

Guia de Uso del Cluster HPC UCR

para CrossSim 2.0 — Proyecto IE0499 I-2026

Reservoir Computing con Materiales 2D para Ingenieria Neuromorfca

Autores:	Ricardo Chacon Carvajal (B11748) Jonathan D. Rodriguez Hernandez (B76490)
Supervisora:	M.Sc. Taina Ramirez Cortes
Fecha:	Mayo 2026
Repositorio:	github.com/jodridez/ProyectoElectrico (rama: Ricardo)
Cluster:	HPC UCR — 172.16.24.2

Este documento describe el flujo completo para ejecutar los experimentos de CrossSim 2.0 en el Cluster HPC institucional de la UCR, desde el acceso remoto hasta la obtencion de resultados con datos experimentales de MoTe_2 del laboratorio CELEQ. Incluye todos los scripts SLURM listos para usar.

1. Contexto Tecnico

1.1 Estado actual del repositorio

El repositorio github.com/jodridez/ProyectoElectrico (rama **Ricardo**) contiene CrossSim 2.0 con las siguientes modificaciones ya aplicadas. No es necesario reaplicarlas:

- **Parche 1** — `inference/inference_util/keras_parser.py`: compatibilidad con `inbound_nodes` de Keras 3.x.
- **Parche 2** — `cross_sim/backprop/backprop.py`: `from warnings import warn` (`NameError` corregido).
- **Parche 3** — `inference/inference_util/inference_net.py`: asignacion de `bp.ndata` en bloque CNN (`ZeroDivisionError` corregido).
- **Parche NumPy** — `cross_sim/backprop/backprop.py` linea 475: `np.array(matrix_list, dtype=object)` para Python 3.12.

Nota: El modulo de inferencia usado es el **nativo de CrossSim 2.0**: `inference/run_inference.py`. Su panel de configuracion es `inference/inference_config.py`. No se usa `applications/dnn/inference/` ni ninguna estructura de CrossSim 3.

1.2 Diferencias clave: Windows vs Cluster

Aspecto	Windows / RTX 4060	Cluster HPC UCR / A100
GPU	RTX 4060, 8 GB VRAM CUDA 13.x	A100-SXM4, 80 GB VRAM CUDA 12.5
TF GPU	NO (limitacion Windows)	SI (Linux nativo)
CuPy	cupy-cuda13x	cupy-cuda12x <-- CAMBIO CRITICO
Ejecucion	python script.py directo en terminal	sbatch script.sh a traves de SLURM
Inferencia	inference/run_inference.py (CrossSim 2.0 + 3 parches)	Igual — mismos archivos y mismos parches

2. Flujo Completo Paso a Paso

FASE 0 — Acceso al cluster

- Conectar **GlobalProtect VPN-UCR** (obligatorio fuera de la red UCR).
- Abrir **MobaXterm** → New Session → SSH → Host: **172.16.24.2**, Port: 22.

IMPORTANTE: NUNCA correr calculos pesados en el nodo de acceso. Solo transferencias, edicion de archivos y envio de jobs con sbatch.

FASE 1 — Subir el repositorio

```
# Clonar directamente en el cluster
git clone https://github.com/jodridez/ProyectoElectrico.git
cd ProyectoElectrico
git checkout Ricardo
git submodule update --init --recursive
```

```
# Verificar quota antes de clonar
quota -s
```

Nota: Quota por defecto: 25 GB. El repo pesa ~3 GB. Si se necesita mas espacio, solicitarlo en <https://soporteci.ucr.ac.cr/>

FASE 2 — Instalar Miniforge3 (solo una vez)

```
wget https://git.ucr.ac.cr/hpc/scripts-instalacion/-/raw/main/miniforge3_install.sh
bash miniforge3_install.sh
```

```
# Reabrir sesion SSH y verificar:
mamba --version
```

FASE 3 — Crear el ambiente conda

IMPORTANTE: El cluster tiene CUDA 12.5. Usar cupy-cuda12x, NO cupy-cuda13x como en Windows.

```
mamba create --name crosssim python=3.12
mamba activate crosssim

pip install "tensorflow[and-cuda]" # detecta CUDA 12.5 automaticamente
pip install cupy-cuda12x           # CUDA 12, no 13
pip install "numpy<2.0" scipy pandas matplotlib pillow
```

Verificar GPU desde sesion interactiva:

```
srun -p gpu -G 1 --pty bash -l

mamba activate crosssim
python3 -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"
# Debe mostrar la A100

python3 -c "import cupy; print(cupy.cuda.runtime.getDeviceCount())"
# Debe mostrar: 1

exit
```

FASE 4 — Smoke test

Verificar que todo funciona antes de enviar jobs largos:

```
srun -p gpu -G 1 --pty bash -l
mamba activate crosssim

# Test training (1 run, 1 epoch)
cd /home/$USER/ProyectoElectrico/training
python MLP_training.py
# Esperado: Numeric ~95%, Single LUT ~92%, Multi LUT ~91%

# Test inferencia
cd /home/$USER/ProyectoElectrico/inference
python run_inference.py
# Esperado: ~80% en condiciones ideales

exit
```

3. Scripts SLURM

Los cuatro scripts a continuacion estan disponibles como archivos .sh independientes. Copiarlos a /home/<user>/ProyectoElectrico/ y sustituir **CORREO** con el correo institucional @ucr.ac.cr antes de usar.

run_training.sh

*Descripcion: Entrenamiento MLP — 3 escenarios: Numeric, Single LUT, Multi LUT | Cola: **gpu** | tiempo maximo: 24 horas*

```
#!/bin/bash -l
#SBATCH --partition=gpu
#SBATCH --time=24:00:00
#SBATCH --nodes=1
#SBATCH --gpus-per-node=1
#SBATCH --ntasks=1
#SBATCH --job-name="crosssim_training"
#SBATCH --output=training_%j.out
#SBATCH --error=training_%j.err
#SBATCH --mail-user=CORREO@ucr.ac.cr
#SBATCH --mail-type=BEGIN,END,FAIL

# Activar ambiente
mamba activate crosssim

# Ir al directorio de training
cd /home/$USER/ProyectoElectrico/training

# Verificar GPU disponible
python3 -c "import tensorflow as tf; print(tf.config.list_physical_devices('GPU'))"

# Ejecutar training
# Parametros en MLP_training.py: Nruns=5, Nepochs=20, batchsize=10
python MLP_training.py
```

run_inference_ideal.sh

Descripción: Inferencia ideal — sin error de programación ni ruido de lectura | Cola: **gpu** | tiempo máximo: 4 horas

```
#!/bin/bash -l
#SBATCH --partition=gpu
#SBATCH --time=4:00:00
#SBATCH --nodes=1
#SBATCH --gpus-per-node=1
#SBATCH --ntasks=1
#SBATCH --job-name="crosssim_inf_ideal"
#SBATCH --output=inference_ideal_%j.out
#SBATCH --error=inference_ideal_%j.err
#SBATCH --mail-user=CORREO@ucr.ac.cr
#SBATCH --mail-type=BEGIN,END,FAIL

# Verificar antes de enviar que inference_config.py tenga:
#   alpha_error = 0.00
#   alpha_noise = 0.00
#   infinite_on_off_ratio = True

mamba activate crosssim
cd /home/$USER/ProyectoElectrico/inference
python run_inference.py
```

run_lut_gen.sh

Descripción: Genera LUT de MoTe2 desde datos I-V del CELEQ (set/reset CSV) | Cola: **serial** | tiempo maximo: **1 hora** | **NO requiere GPU**

```
#!/bin/bash -l
#SBATCH --partition=serial
#SBATCH --time=1:00:00
#SBATCH --ntasks=1
#SBATCH --job-name="lut_gen_mote2"
#SBATCH --output=lut_gen_%j.out
#SBATCH --error=lut_gen_%j.err
#SBATCH --mail-user=CORREO@ucr.ac.cr
#SBATCH --mail-type=END, FAIL

# Antes de enviar:
# 1. Subir set_MoTe2.csv y reset_MoTe2.csv al cluster:
#   scp set_MoTe2.csv reset_MoTe2.csv USER@172.16.24.2:/home/USER/
#       ProyectoElectrico/examples/lookup_table_generation/
#
# 2. Editar create_lookup_table.py:
#   folder      = 'MoTe2_LUT'
#   set_file     = 'set_MoTe2.csv'
#   reset_file  = 'reset_MoTe2.csv'
#   read_voltage = <Vread del equipo CELEQ en V>
#               (si datos en conductancia S, usar read_voltage = 1)

mamba activate crosssim
cd /home/$USER/ProyectoElectrico/examples/lookup_table_generation

# Verificar que los CSV existen
[ ! -f "set_MoTe2.csv" ]    && echo "ERROR: set_MoTe2.csv no encontrado"    && exit 1
[ ! -f "reset_MoTe2.csv" ] && echo "ERROR: reset_MoTe2.csv no encontrado" && exit 1

python create_lookup_table.py

# Verificar salida
ls -lh MoTe2_LUT/
# Debe mostrar dG_increasing.txt y dG_decreasing.txt
```

run_inference_nonideal.sh

Descripción: Inferencia con no-idealidades reales del dispositivo MoTe2 | Cola: gpu | tiempo maximo: 4 horas

```
#!/bin/bash -l
#SBATCH --partition=gpu
#SBATCH --time=4:00:00
#SBATCH --nodes=1
#SBATCH --gpus-per-node=1
#SBATCH --ntasks=1
#SBATCH --job-name="crosssim_inf_mote2"
#SBATCH --output=inference_mote2_%j.out
#SBATCH --error=inference_mote2_%j.err
#SBATCH --mail-user=CORREO@ucr.ac.cr
#SBATCH --mail-type=BEGIN,END,FAIL

# Antes de enviar, configurar inference/inference_config.py:
#
# Rmin = <valor CELEQ en ohms – estado LRS>
# Rmax = <valor CELEQ en ohms – estado HRS>
# infinite_on_off_ratio = False
#
# error_model = "generic"
# alpha_error = <derivado del analisis estadistico de la LUT MoTe2>
#
# noise_model = "generic"
# alpha_noise = <de mediciones C2C, o 0.00 si no disponible>
#
# drift_model = "none"
# t_drift = 0
#
# Referencia Ion/Ioff ~ 10^3 (Rupom et al. 2025):
# Rmin ~ 1e3 ohm Rmax ~ 1e6 ohm

mamba activate crosssim
cd /home/$USER/ProyectoElectrico/inference
python run_inference.py
```


4. Orden de Ejecucion

Comandos para enviar cada script a cola, en el orden correcto:

```
# Ir al directorio del proyecto
cd /home/$USER/ProyectoElectrico

# --- ETAPA 1: Baseline con material de referencia ---
sbatch run_training.sh
squeue --me                # verificar que entro a cola
tail -f training_<JOBID>.out # monitorear en tiempo real

# --- ETAPA 2: Inferencia ideal (una vez terminado el training) ---
sbatch run_inference_ideal.sh

# --- ETAPA 3: Cuando lleguen los datos del CELEQ ---
# Subir los CSV primero:
# scp set_MoTe2.csv reset_MoTe2.csv USER@172.16.24.2:/home/USER/
# ProyectoElectrico/examples/lookup_table_generation/
sbatch run_lut_gen.sh

# --- ETAPA 4: Training con LUT de MoTe2 ---
# (despues de revisar las graficas de verificacion del LUT)
sbatch run_training.sh

# --- ETAPA 5: Inferencia con hardware no ideal ---
# (despues de configurar Rmin/Rmax en inference_config.py)
sbatch run_inference_nonideal.sh
```

Monitoreo de jobs

```
squeue --me                # estado de mis jobs
ssh -t cngpu001 nvidia-smi # uso GPU en tiempo real
tail -f training_<JOBID>.out # output en vivo
scancel <JOBID>             # cancelar si es necesario
reportseff -u $USER         # eficiencia al terminar
```

Bajar resultados al equipo local

```
# Desde MobaXterm o PowerShell en Windows:
scp -r USER@172.16.24.2:/home/USER/ProyectoElectrico/training/sweep_results .
scp -r USER@172.16.24.2:/home/USER/ProyectoElectrico/training/cross_sim_models .
scp USER@172.16.24.2:/home/USER/ProyectoElectrico/*.out .
```

5. Referencia Rapida

Comandos SLURM esenciales

Comando	Descripcion
<code>sbatch <script.sh></code>	Enviar job a cola
<code>squeue --me</code>	Ver mis jobs en cola (columna ST = estado)
<code>scancel <JOBID></code>	Cancelar un job
<code>scontrol show job <JOBID></code>	Detalle de job en ejecucion
<code>sacct -j <JOBID></code>	Info de job ya terminado
<code>reportseff -u \$USER</code>	Eficiencia de CPU/GPU de mis jobs
<code>srunch -p gpu -G 1 --pty bash -l</code>	Sesion interactiva en nodo GPU
<code>srunch -n1 --pty bash -l</code>	Sesion interactiva en nodo CPU
<code>quota -s</code>	Ver espacio disponible en /home

Estados de un job (columna ST en squeue)

ST	Estado	Que significa
R	RUNNING	Corriendo en un nodo
PD	PENDING	Esperando recursos — eventualmente corre
CD	COMPLETED	Finalizo exitosamente
F	FAILED	Termino con error (revisar archivo .err)
CA	CANCELLED	Cancelado manualmente
TO	TIMEOUT	Supero el tiempo limite del QoS
OOM	OUT_OF_MEM	Error de memoria

Resultados de referencia (Windows / RTX 4060, 1 epoch)

Experimento	Escenario	Precision
Training defecto (DWMTJ)	Numeric (ideal)	94.96%
Training defecto (DWMTJ)	Single LUT	92.41%
Training defecto (DWMTJ)	Multi LUT	90.91%
Training alpha x4 (DWMTJ)	Numeric	96.36%
Training alpha x4 (DWMTJ)	Single LUT	94.88%

Training alpha x4 (DWMJTJ)	Multi LUT	92.89%
Inferencia CrossSim 2.0 nativo	Condiciones ideales	~80%